Especificación de Requisitos del Software (ERS)

Sistema Ch.Store — Tienda online (React, API REST y base de datos)

Asignatura: DSY1104 — Desarrollo Full Stack II
Evaluación: Parcial 2 (30%)
Versión: 2.1 (propuesta terminada)
Fecha: 28 de octubre de 2026

Ficha del documento

Fecha	Revisión	Autor	Modificación
30-09-2026	1.0	Javiera Rojas	Versión inicial del documento. Secciones 1, 2 y 3 desarrolladas para la entrega 1.
19-10-2026	1.1	Javiera Rojas, Mhyjal Rozas	Integración del diseño visual de la tienda con la lógica JavaScript. Inicio de sesión con perfil, catálogo único para tienda y panel, y registro de órdenes.
28-10-2026	2.0	Javiera Rojas, Mhyjal Rozas	Migración del frontend a React con Bootstrap. Nuevas vistas de la tienda (categorías, ofertas, checkout, pago exitoso y pago con error) y del panel (órdenes y boleta, productos críticos, categorías, historial de compras, reportes y perfil). Fuente de datos simulada con CRUD y persistencia. Estrategia de pruebas con Jasmine y Karma (sección 3.5). Se reescriben las restricciones y los requisitos no funcionales afectados.
28-10-2026	2.1	Javiera Rojas, Mhyjal Rozas	Integración con un backend propio: API REST con Express y base de datos SQLite, autenticación real con contraseñas cifradas y sesiones con token, permisos por perfil validados en el servidor y descuento de stock al registrar la orden. Se agregan los requisitos RF-28 a RF-35 y RNF-22 a RNF-26, se actualizan las restricciones, las interfaces y la estrategia de pruebas (frontend y backend).

> **Alcance de esta versión:** el documento queda terminado para la entrega 2. El frontend en React funciona en dos modos: con el **backend** propio (API REST y base de datos SQLite, con autenticación real) o, si el servidor no está disponible, en **modo local**, con los datos simulados en JavaScript y guardados en el navegador. Las interfaces de software y de comunicación con el backend se describen en 3.1.3 y 3.1.4.

---

1. Introducción

1.1 Propósito

Especificar los requisitos del sistema **Ch.Store**, una tienda online de ropa femenina, en su versión 2: describe qué debe hacer el sistema, bajo qué restricciones y con qué criterios se considerará satisfecho cada requisito, incluida la estrategia de pruebas que lo verifica.

Está dirigido al **equipo de desarrollo**, al **cliente** (Ch.Store) y al **docente de la asignatura**.

1.2 Ámbito del Sistema

Qué hará el sistema:

- Exhibir un catálogo de productos, con búsqueda, filtros por categoría y por stock, orden por precio, vista de categorías y vista de ofertas.
- Permitir armar un carrito (con talla, cupones y persistencia) y completar una compra en un paso de pago con datos del cliente y dirección de entrega.
- Mostrar el resultado del pago (exitoso, con boleta, o con error y posibilidad de reintento).
- Permitir el registro y el inicio de sesión de usuarios; con el backend, la contraseña y la sesión las valida el servidor.
- Guardar los datos en una base de datos y exponerlos mediante una API REST, con permisos según el perfil.
- Recibir consultas por un formulario de contacto y publicar artículos de blog.
- Entregar al personal un panel de administración para mantener productos, categorías y usuarios, consultar órdenes y boletas, ver el historial de compras de cada usuario, detectar productos en stock crítico y revisar reportes.
- Restringir las funciones del panel según el perfil (Administrador o Vendedor).

Qué no hará el sistema en esta versión:

- No procesa pagos reales: el resultado del pago se simula.
- No exige un servidor para funcionar: si el backend no responde, la tienda usa datos de ejemplo guardados en `localStorage` (modo local).
- No envía correos; "enviar boleta por email" abre el cliente de correo del usuario.
- En modo local no hay autenticación real: la sesión se identifica por el correo y la contraseña solo se valida en formato.

Objetivos:

Objetivo	Meta
Ofrecer una experiencia de compra completa de punta a punta	Del catálogo a la boleta sin salir de la aplicación
Centralizar la gestión del negocio	Un único panel para productos, categorías, usuarios y órdenes
Disminuir los quiebres de stock	Alerta y listado de productos críticos
Asegurar la calidad del sistema	Pruebas unitarias automatizadas con cobertura superior al 90% de líneas en el frontend y en el backend
Proteger la información	Contraseñas cifradas, sesión con token y permisos por perfil validados en el servidor

1.3 Definiciones, Acrónimos y Abreviaturas

Término	Definición
ERS	Especificación de Requerimientos de Software.
SPA	Aplicación de una sola página: el navegador carga una vez la aplicación y la navegación ocurre sin recargar.
Componente	Función de React que recibe propiedades (props) y devuelve parte de la interfaz.
Props / Estado	Datos que un componente recibe de su padre (props) y datos que administra por sí mismo (estado).
Contexto	Mecanismo de React para compartir estado (carrito, sesión) entre componentes sin pasarlo por props.
Bootstrap	Framework CSS que aporta el sistema de grilla y los componentes responsivos.
Jasmine / Karma	Framework de pruebas (Jasmine) y ejecutor que las corre en un navegador real (Karma).
Mock / Espía (spy)	Reemplazo controlado de una dependencia para probar un componente de forma aislada.

Término	Definición
Cobertura	Porcentaje del código fuente ejecutado por las pruebas.
RUN	Rol Único Nacional. Su último carácter es el dígito verificador (módulo 11).
Stock crítico	Cantidad a partir de la cual un producto necesita reposición.
Cupón	Código que aplica un descuento porcentual sobre el subtotal.
API REST	Servicio web que expone los datos por HTTP con rutas como `/api/productos` y respuestas en JSON.
Backend	Programa servidor (Node.js con Express) que guarda los datos en la base y aplica la seguridad.
SQLite	Base de datos relacional que vive en un solo archivo y no requiere instalar un servidor de base de datos.
Hash / scrypt	Transformación irreversible que se aplica a las contraseñas antes de guardarlas; scrypt es el algoritmo usado.
Token de sesión	Código aleatorio que el servidor entrega al iniciar sesión y que el navegador envía en cada petición para identificarse.
Modo API / modo local	Formas de operar de la capa de datos: con el backend, o con `localStorage` si el servidor no responde.

1.4 Referencias

Documento	Fuente
DSY1104 Evaluación Parcial 2 — Anexo 1: Instrucciones	Escuela de Administración y Negocios
DSY1104 Evaluación Parcial 2 — Pauta de evaluación	Subdirección de Evaluación de Resultados de Aprendizaje
ERS Ch.Store versión 1.1 (`docs/ERS-v1.md`)	Equipo de desarrollo
Documento de cobertura del testing (`docs/cobertura-testing.md`)	Equipo de desarrollo
Documentación del backend (`backend/README.md`)	Equipo de desarrollo
IEEE Std 830-1998	IEEE
Documentación de React, React Router, Bootstrap, Jasmine y Karma	Sitios oficiales

1.5 Visión General del Documento

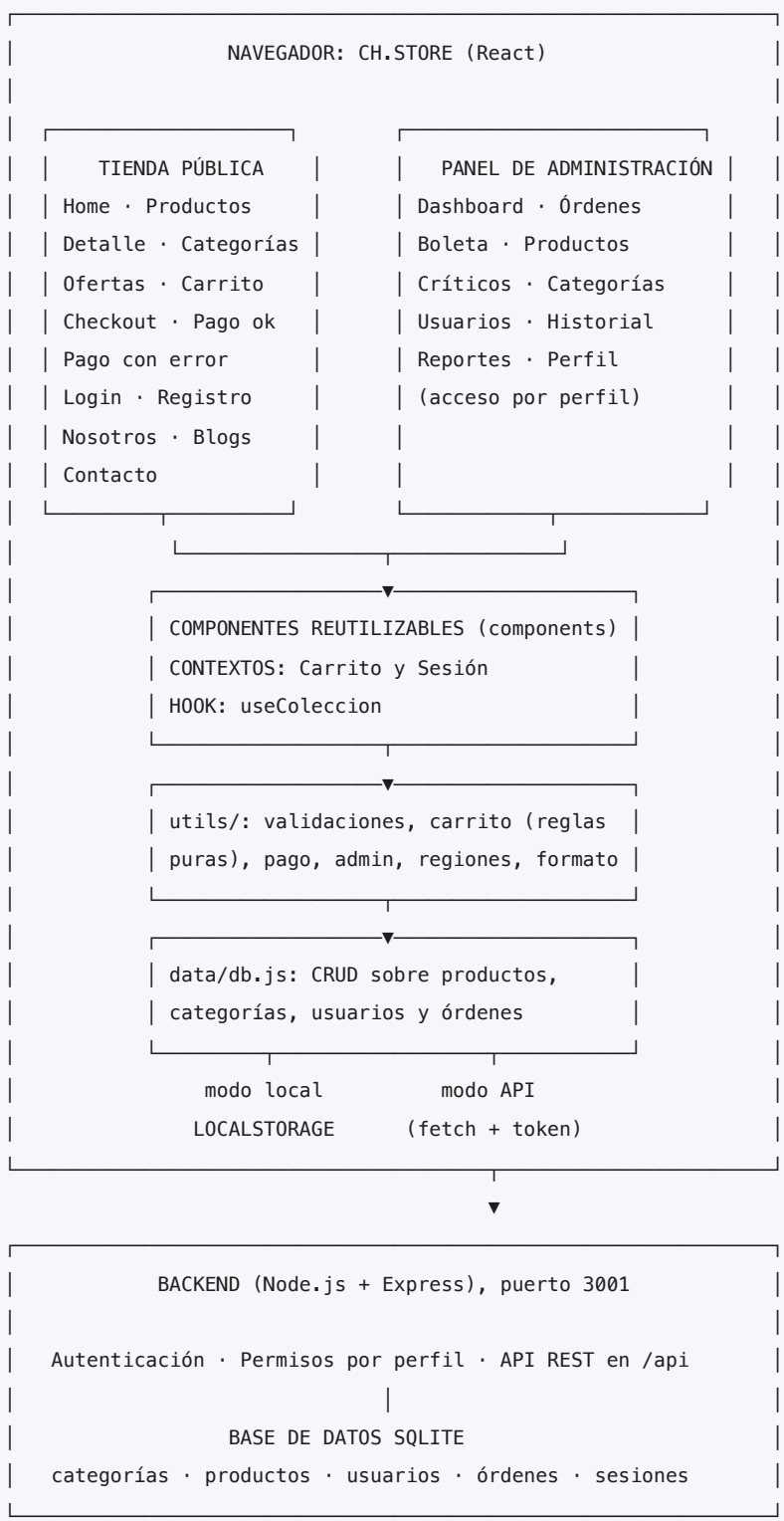
La **sección 1** introduce el propósito y el alcance. La **sección 2** describe el contexto del producto. La **sección 3** contiene los requisitos específicos: interfaces, funcionales, no funcionales, otros requisitos y la estrategia de pruebas.

---

2. Descripción General

2.1 Perspectiva del Producto

Ch.Store es un producto independiente. En la versión 2 es una **aplicación de una sola página (SPA)** construida con React, con dos áreas que comparten componentes, datos y reglas, y un **backend** propio que guarda los datos y aplica la seguridad:



Arquitectura de componentes. Cada componente tiene una única responsabilidad:

Capa	Archivos	Responsabilidad
Presentación reutilizable	`components/` (`Boton`, `CampoFormulario`, `ProductCard`, `ProductList`, `Precio`, `TablaItems`, `FilaCarrito`, `ResumenTotales`, `SelectorRegionComuna`, `Alerta`, `MensajeError`, `ConfirmarEliminar`, `TarjetaIndicador`)	Reciben datos y funciones por props; solo manejan estado cuando es necesario
Estructura	`Navbar`, `Footer`, `Buscador`, `LayoutTienda`, `LayoutAdmin`, `RutaAdmin`	Encabezado, pie, menú lateral y control de acceso

Capa	Archivos	Responsabilidad
Vistas	`pages/` y `pages/admin/`	Una vista por ruta; combinan componentes y datos
Estado compartido	`context/`	Carrito y cupón; sesión y perfil
Reglas de negocio	`utils/`	Funciones puras, sin DOM ni almacenamiento, fáciles de probar
Datos (frontend)	`data/db.js`	Única puerta de entrada a los datos: CRUD, consultas y suscripción a cambios; opera en modo local (`localStorage`) o en modo API (caché en memoria y escritura en el servidor)
Backend	`backend/src/` (`app.js`, `repositorio.js`, `db.js`, `seguridad.js`, `sesiones.js`)	API REST con Express, esquema y datos iniciales de SQLite, autenticación, sesiones y permisos por perfil

Flujos de navegación (según las Figuras 2, 3 y 10 de las instrucciones):

- Tienda: Home → Productos → Detalle → Carrito → Checkout → Pago exitoso / Pago con error; Categorías → Categoría; Ofertas; Registro / Login; Nosotros; Blogs → Detalle; Contacto.
- Panel: Dashboard → Órdenes → Boleta; Productos → Nuevo / Editar / Críticos; Categorías → Nueva / Editar; Usuarios → Nuevo / Editar / Historial de compras; Reportes; Perfil.

2.2 Funciones del Producto

Módulo	Función	Descripción resumida
Catálogo	Listar y filtrar	Búsqueda por texto, categoría, solo con stock y orden por precio
Catálogo	Ver detalle	Ficha con galería, talla, cantidad y botón de compra
Catálogo	Categorías y ofertas	Vistas que separan los productos por categoría y muestran los que tienen descuento
Compra	Gestionar carrito	Agregar, cambiar cantidades, eliminar, vaciar y aplicar cupón
Compra	Checkout	Datos del cliente y dirección de entrega; se autocompletan con la sesión
Compra	Resultado del pago	Boleta de compra exitosa o pantalla de pago con error y reintento
Cuentas	Registro e inicio de sesión	Validaciones completas; con el backend, contraseña cifrada y sesión con token
Comunicación	Contacto y blog	Formulario validado y artículos con detalle
Administración	Mantenedores	Productos, categorías y usuarios con CRUD completo
Administración	Órdenes y boletas	Listado con filtro por estado, boleta imprimible y cambio de estado
Administración	Críticos, historial y reportes	Productos por reponer, compras por usuario, ingresos y más vendidos
Seguridad	Control de acceso	Rutas y menú según el perfil activo; el servidor valida el token y el perfil en cada operación
Datos	Persistencia	API REST sobre una base SQLite, con respaldo local si el servidor no responde

2.3 Características de los Usuarios

Perfil	Experiencia	Responsabilidad	Frecuencia
Administrador	Usuario intermedio, planillas	Acceso total: productos, categorías, usuarios, órdenes y reportes	Diaria
Vendedor	Usuario básico	Consulta productos y gestiona las órdenes (las consulta y cambia su estado); no modifica el catálogo, las categorías ni los usuarios	Diaria
Cliente	Usuario habitual de comercio electrónico y móvil	Navega, compra y revisa su sesión	Ocasional

2.4 Restricciones

Ámbito	Restricción
Tecnología	Frontend con **React** (Vite) y **Bootstrap 5** para el diseño responsivo; enrutamiento con React Router. Backend con **Node.js 22.13 o superior**, **Express** y **SQLite** mediante el módulo `node:sqlite`.
Datos	Los componentes acceden a los datos solo por `data/db.js`. Con el backend se guardan en la base SQLite (`backend/data/chstore.db`, creada y poblada al arrancar); sin él, en `localStorage`.
Pruebas	Las pruebas unitarias se escriben con **Jasmine** y se ejecutan con **Karma** en Chrome.
Hardware	Debe operar en equipos de gama baja y teléfonos móviles.
Navegadores	Versiones actuales de Chrome, Firefox y Edge.
Control de versiones	Repositorio público de GitHub con commits descriptivos y aporte verificable de cada integrante.
Seguridad	Con el backend, la autenticación y los permisos los aplica el servidor; las validaciones y el control de acceso del cliente solo mejoran la usabilidad. En modo local son únicamente preventivos.
Despliegue	Todo el sistema se levanta con `npm run dev` desde la raíz del repositorio; no hay que instalar un motor de base de datos.
Criticidad	No se manejan datos financieros ni sensibles.

2.5 Suposiciones y Dependencias

- Los usuarios acceden desde un navegador con JavaScript y `localStorage` habilitados.
- Las imágenes de los productos se alojan en servicios externos mediante URL; el logo y el video de portada están en `public/img/`.
- El listado de regiones y comunas se mantiene estable (`utils/regiones.js`).
- Dependencias de ejecución del frontend: `react`, `react-dom`, `react-router-dom`, `bootstrap` y `bootstrap-icons`; del backend: `express`. Dependencias de prueba: `jasmine-core`, `karma`, `karma-jasmine`, `karma-chrome-launcher`, `karma-webpack`, `karma-coverage`, `karma-spec-reporter`, `webpack` y Babel.
- Se necesita Node.js 22.13 o superior para instalar, compilar y ejecutar el backend, y Google Chrome para ejecutar las pruebas del frontend.

2.6 Requisitos Futuros

1. **Pasarela de pago** real que reemplace el resultado simulado, con validación en el servidor de los precios y totales de cada orden.
2. **Envío de boletas por correo** desde el servidor.

3. **Carga de imágenes** desde el panel.
4. **Reportes exportables** y gráficos.
5. **Pruebas de extremo a extremo** automatizadas y accesibilidad WCAG 2.1 AA.
6. **Endurecer la seguridad:** HTTPS, token en una cookie `HttpOnly`, reglas más exigentes para las contraseñas y recuperación de contraseña.

---

3. Requisitos Específicos

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaces de usuario

Tienda pública: barra superior con logo, buscador, menú (Home, Productos, Categorías con desplegable, Ofertas, Nosotros, Blog, Contacto), botón de carrito con total y número de unidades, y accesos de sesión. En pantallas pequeñas el menú se colapsa en un botón hamburguesa. Contenido central en contenedor Bootstrap y pie con enlaces secundarios.

Panel de administración: barra superior oscura, menú lateral según el rol y área de contenido. En móvil el menú lateral pasa a ser desplegable. Listados en tablas responsivas y altas/ediciones en formularios.

Convenciones comunes:

Elemento	Definición
Sistema de diseño	Bootstrap 5 y Bootstrap Icons
Grilla de productos	1 columna en móvil, 2 en tablet, 4 en escritorio
Campos obligatorios	Asterisco rojo junto a la etiqueta
Mensajes de error	Bajo el campo afectado (`is-invalid` / `invalid-feedback`), nunca en ventanas emergentes
Confirmaciones	Avisos (`alert`) con color según el resultado; las eliminaciones piden confirmación
Impresión	La boleta se imprime ocultando encabezado, menús y botones

3.1.2 Interfaces de hardware

Computadores desde 1024 px de ancho, tablets en ambas orientaciones y teléfonos desde 360 px. Los controles deben poder accionarse con pantalla táctil.

3.1.3 Interfaces de software

Producto	Propósito	Interfaz
Navegador web (`localStorage`)	Persistir datos en modo local, el carrito y la sesión	JSON bajo las claves `chstore_productos`, `chstore_categorias`, `chstore_usuarios`, `chstore_ordenes` (solo en modo local), `chstore_carrito`, `chstore_cupon`, `chstore_sesion` y `chstore_token`
Backend Ch.Store	Leer y guardar los datos, autenticar y autorizar	API REST bajo `/api` en el puerto 3001, con cuerpos en JSON
SQLite (`node:sqlite`)	Persistir los datos del servidor	Archivo `backend/data/chstore.db`, con las tablas `categorias`, `productos`, `usuarios`, `ordenes` y `sesiones`

Producto	Propósito	Interfaz
Cliente de correo del usuario	Enviar la boleta	Enlace `mailto:` con el correo del cliente y el número de orden

3.1.4 Interfaces de comunicación

El frontend se comunica con el backend por HTTP, con cuerpos en JSON. En desarrollo, Vite reenvía las rutas `/api` al puerto 3001, por lo que no se necesita configurar CORS. Las peticiones de un usuario con sesión llevan el encabezado `Authorization: Bearer <token>`.

Ruta	Método	Función
`/api/auth/login`, `/api/auth/registro`	POST	Iniciar sesión y registrar un cliente
`/api/auth/logout`, `/api/auth/yo`	POST, GET	Cerrar la sesión y consultar al usuario de la sesión
`/api/:coleccion`	GET, POST	Listar y crear en `productos`, `categorias`, `usuarios` u `ordenes`
`/api/:coleccion/:id`	GET, PUT, DELETE	Leer, actualizar y eliminar un registro
`/api/reiniciar`, `/api/salud`	POST, GET	Restaurar los datos iniciales (administrador) y comprobar el servidor

Códigos de respuesta: `401` sin sesión, `403` sin permiso, `404` registro inexistente, `409` conflicto (dato repetido o relación que impide la operación), `429` demasiados intentos de inicio de sesión. No hay comunicación con otros sistemas.

3.2 Requisitos funcionales

La columna **Pruebas** indica dónde se verifica cada requisito (detalle en el documento de cobertura). Las abreviaturas son: **VAL** `validaciones.spec.js`, **CAR** `carrito.spec.js`, **DB** `db.spec.js`, **COMP** `componentes.spec.jsx`, **PAG** `paginas.spec.jsx`, **ADM** `admin.spec.js`, **ADP** `admin-paginas.spec.jsx`, **NAV** `navegacion.spec.jsx`, **API** `db-api.spec.js`, **AUT** `auth-api.spec.jsx` y **BE** `backend/test/api.test.js`.

Tienda

ID	Requisito	Entradas	Proceso y salidas	Situaciones anormales	Pruebas
RF-01	**Listar y filtrar el catálogo.**	Texto de búsqueda (`?q=`), categoría, "solo con stock", orden por precio	Se filtra el catálogo y se muestra una tarjeta por producto en una grilla responsiva	Sin coincidencias: "No hay productos que coincidan con los filtros."	PAG, COMP, NAV
RF-02	**Mostrar el detalle de un producto.**	Código en la ruta `/producto/:codigo`	Ficha con galería, categoría, descripción, precio con oferta, stock, talla y cantidad	Código inexistente: aviso y enlace al catálogo	PAG, NAV
RF-03	**Agregar al carrito.**	Código, talla (S, M, L) y cantidad	Reglas R1–R4: sin stock no se agrega, cantidad mínima 1, tope por stock sumando tallas, misma talla suma cantidades	Producto agotado: botón deshabilitado. Cantidad sobre el stock: se ajusta y se informa	CAR, PAG

ID	Requisito	Entradas	Proceso y salidas	Situaciones anormales	Pruebas
RF-04	**Persistir el carrito.**	Contenido del carrito	Se guarda en `localStorage` en cada cambio y se recupera al cargar	Dato corrupto: carrito vacío sin interrumpir la navegación	PAG, NAV
RF-05	**Modificar el carrito.**	Línea y acción	Sumar, restar (al llegar a 0 se elimina), eliminar y limpiar; totales recalculados	Cantidad máxima = stock disponible	CAR, PAG
RF-06	**Aplicar un cupón.**	Código	Se normaliza a mayúsculas; un solo cupón activo; descuento porcentual sobre el subtotal	Cupón inexistente o carrito vacío: aviso sin cambiar el total	CAR, PAG
RF-07	**Registrar un usuario.**	RUN, nombre, apellidos, correo, contraseña y confirmación, teléfono, región, comuna, dirección	Validación por campo; se guarda con perfil Cliente. Con el backend la contraseña viaja al servidor, que la guarda cifrada; en modo local no se guarda	RUN o correo duplicados se informan en el campo	VAL, DB, PAG, API, AUT, BE
RF-08	**Validar el RUN.**	RUN sin puntos ni guion, 7 a 9 caracteres	Módulo 11 con serie 2–7; 11 equivale a 0 y 10 a K	Mensaje distinto para formato, largo y dígito verificador	VAL
RF-09	**Iniciar y cerrar sesión.**	Correo y contraseña (4 a 10 caracteres)	Con el backend, el servidor valida el correo y la contraseña y entrega un token; en modo local se identifica al usuario por su correo. Se guarda la sesión con su perfil; Administrador y Vendedor van al panel	Formato inválido, credenciales incorrectas o bloqueo temporal: error en pantalla	PAG, NAV, API, AUT, BE
RF-10	**Enviar un mensaje de contacto.**	Nombre, correo (opcional), comentario (máx. 500)	Validación y contador de caracteres; confirmación y limpieza	Campos inválidos impiden el envío	PAG
RF-11	**Listar categorías.**	Ruta `/categorias` o `/categorias/:nombre`	Tarjetas de categorías y sus productos; con nombre, solo esa categoría	Categoría inexistente: aviso	NAV
RF-12	**Mostrar ofertas.**	—	Lista los productos con descuento mayor a 0, con precio original tachado y precio final	Sin ofertas: mensaje	PAG, COMP, DB
RF-13	**Realizar el checkout.**	Nombre, apellidos, correo, calle, departamento (opcional), región, comuna, indicaciones (opcional)	Resumen del carrito y formulario validado; los datos se autocompletan si hay sesión; comuna en cascada según la región	Carrito vacío: no permite pagar. Campos inválidos: no se registra la orden	PAG, ADM
RF-14	**Confirmar la compra y mostrar su resultado.**	Compra con resultado simulado (aprobado o rechazado)	Aprobado: código `CH–XXXXXX`, orden registrada, stock descontado, carrito vaciado y boleta con opción de imprimir y enviar por correo. Rechazado: intento registrado como "Pago rechazado", carrito y stock intactos y botón "Volver a realizar el pago". Con el backend, la orden se guarda en el servidor, que descuenta el stock (RF-32)	Orden inexistente: aviso	PAG, CAR, API, BE

ID	Requisito	Entradas	Proceso y salidas	Situaciones anormales	Pruebas
RF-15	**Mostrar nosotros y blogs.**	—	Páginas informativas y artículos con listado, detalle y siguiente lectura	Artículo inexistente: aviso	NAV

Panel de administración

ID	Requisito	Entradas	Proceso y salidas	Situaciones anormales	Pruebas
RF-16	**Controlar el acceso por perfil.**	Perfil de la sesión	Sin sesión o con perfil Cliente: redirige al login con aviso. El menú y las rutas dependen del rol (Vendedor: Dashboard, Productos, Órdenes y Perfil). El servidor rechaza con 401 o 403 las operaciones no permitidas (RF-29)	Vendedor en una sección reservada: vuelve al Dashboard	ADM, PAG, ADP, NAV, AUT, BE
RF-17	**Mostrar el dashboard.**	Datos del sistema	Indicadores de compras, ingresos, productos, inventario y usuarios; alerta de stock crítico y accesos por rol	—	ADP, ADM
RF-18	**Mantener productos.**	Código, nombre, descripción, precio, stock, stock crítico, descuento, categoría, imagen	Alta, edición (código bloqueado), eliminación con confirmación y búsqueda; solo el Administrador modifica	Código duplicado, categoría inexistente o campos inválidos: no se guarda	ADP, ADM, DB, API, BE
RF-19	**Listar productos críticos.**	Catálogo	Productos con stock igual o menor al crítico, con estado Normal, Crítico o Agotado	—	ADP, DB, ADM
RF-20	**Mantener categorías.**	Nombre y descripción	Alta, edición (nombre bloqueado) y eliminación	No se elimina una categoría con productos	ADP
RF-21	**Mantener usuarios.**	RUN, nombre, apellidos, correo, tipo, fecha de nacimiento, región, comuna, dirección	Alta, edición (RUN bloqueado), eliminación, búsqueda y filtro por perfil. Con el backend, la contraseña es obligatoria al crear y opcional al editar (la restablece)	RUN o correo repetidos; no se elimina la propia cuenta	ADP, ADM, AUT, BE
RF-22	**Consultar órdenes y boletas.**	Filtro por estado	Listado y boleta con detalle; el Administrador y el Vendedor pueden cambiar el estado de una orden pagada	Orden de ejemplo sin detalle: se informa	ADP, BE
RF-23	**Ver el historial de compras de un usuario.**	RUN	Órdenes asociadas al correo del usuario y total comprado	Usuario sin compras: mensaje	ADP, DB
RF-24	**Mostrar reportes.**	Órdenes y productos	Ingresos, ticket promedio, productos críticos, más vendidos y órdenes por estado; los pagos rechazados no cuentan como venta	Sin ventas con detalle: mensaje	ADP, ADM
RF-25	**Editar el perfil propio.**	Nombre, apellidos y dirección	Validación y guardado en la base de datos; el servidor no permite cambiar el RUN, el correo ni el perfil de acceso desde esta pantalla	Campos vacíos: no se guarda	ADP, BE

Datos

ID	Requisito	Descripción	Pruebas
RF-26	**Fuente de datos con CRUD.**	`data/db.js` ofrece `listar`, `obtener`, `crear`, `actualizar`, `eliminar` y `reiniciar` para productos, categorías, usuarios y órdenes; rechaza identificadores repetidos y notifica a los componentes suscritos tras cada cambio para que la interfaz se actualice; con el backend, lee de una caché en memoria y envía cada cambio al servidor	DB, API
RF-27	**Persistencia.**	Cada colección se guarda en la base SQLite del backend o, en modo local, en `localStorage`; si el dato local es inválido se usan los datos iniciales y se registra el error	DB, API, BE

Backend y seguridad

ID	Requisito	Descripción	Pruebas
RF-28	**Autenticar usuarios en el servidor.**	`POST /api/auth/login` recibe correo y contraseña, los compara con el hash guardado y entrega un token y los datos del usuario, sin la contraseña. Un correo inexistente y una contraseña incorrecta dan el mismo mensaje	BE, API, AUT
RF-29	**Autorizar por perfil en cada operación.**	Catálogo: lectura pública y escritura solo del Administrador. Usuarios: solo el Administrador, salvo la cuenta propia. Órdenes: cualquiera puede comprar; solo Administrador y Vendedor las ven y cambian su estado; solo el Administrador las elimina. Sin sesión responde 401 y sin permiso 403	BE
RF-30	**Gestionar las sesiones.**	El token vence a las 8 horas, se invalida al cerrar sesión o al eliminar al usuario y se guarda en la base solo como hash. Si el servidor lo rechaza, la aplicación cierra la sesión en pantalla	BE, API, AUT
RF-31	**Limitar los intentos de inicio de sesión.**	Cinco fallos seguidos con el mismo correo desde la misma dirección bloquean el login durante 15 minutos (429 con `Retry-After`); un acceso correcto reinicia el contador	BE
RF-32	**Registrar órdenes y descontar el stock en el servidor.**	Al crear una orden aprobada, el servidor descuenta el stock en la misma transacción y nunca lo deja bajo cero. Quien no es personal solo puede crear órdenes aprobadas o rechazadas	BE, API
RF-33	**Mantener la integridad de los datos.**	Identificadores y correos únicos (sin distinguir mayúsculas), producto con categoría existente y categoría con productos que no se elimina; los conflictos responden 409 con un mensaje	BE
RF-34	**Respaldo local.**	Si el backend no responde al abrir la aplicación, el frontend usa los datos de ejemplo en `localStorage` y sigue funcionando	API
RF-35	**Cargar solo los datos permitidos.**	Tras iniciar sesión, el frontend descarga lo que el perfil puede ver: el Administrador, usuarios y órdenes; el Vendedor, órdenes; el Cliente, solo su cuenta	API, AUT

3.3 Requisitos no funcionales

3.3.1 Rendimiento

ID	Requisito
RNF-01	El 95% de las vistas debe renderizarse completamente en menos de 2 segundos en una conexión de 10 Mbps.
RNF-02	Las operaciones del carrito deben reflejarse en pantalla en menos de 200 ms.
RNF-03	El filtrado debe mantenerse bajo 500 ms con hasta 500 productos.

3.3.2 Seguridad

ID	Requisito
RNF-04	Las rutas del panel deben protegerse según el perfil mediante `RutaAdmin` en el frontend y mediante la validación del token y del perfil en cada ruta de la API, no solo ocultando opciones.
RNF-05	Los datos de entrada deben validarse antes de procesarse, con las reglas centralizadas en `utils/validaciones.js`.
RNF-06	No deben almacenarse contraseñas en el navegador ni exponerse en la URL; en el servidor solo se guarda su hash.
RNF-07	Los errores de lectura del almacenamiento se registran en consola y el sistema continúa con datos iniciales.

| RNF-22 | Las contraseñas deben guardarse con scrypt y una sal distinta por usuario, y la API nunca debe devolverlas. |

| RNF-23 | El token de sesión debe ser aleatorio, de 256 bits, guardarse en la base solo como hash y vencer a las 8 horas. |

| RNF-24 | El servidor debe bloquear el inicio de sesión tras 5 fallos seguidos durante 15 minutos y responder igual ante un usuario inexistente que ante una contraseña mala. |

| RNF-25 | El registro público solo debe poder crear clientes; un usuario sin perfil de administrador no puede cambiar su perfil de acceso, su correo ni su RUN. |

| RNF-26 | El servidor debe validar los permisos aunque el frontend oculte las opciones. |

> **Observación:** en modo local no hay servidor, por lo que el control de acceso y las validaciones del cliente mejoran la usabilidad pero no constituyen seguridad efectiva. La seguridad real depende de usar el backend. Límites conocidos del backend: el pago es simulado y el servidor confía en los precios y totales que envía el frontend; el token se guarda en `localStorage` y no hay HTTPS en desarrollo; la regla de contraseña (4 a 64 caracteres) es la del curso y es débil para un sistema real.

3.3.3 Fiabilidad y disponibilidad

ID	Requisito
RNF-08	Un fallo de `localStorage` no debe interrumpir la navegación.
RNF-09	El sistema no debe mostrar errores en la consola durante la operación normal.
RNF-10	El carrito, el catálogo y las órdenes deben funcionar sin conexión, una vez cargada la aplicación, y la aplicación debe abrirse en modo local si el servidor no responde.

3.3.4 Mantenibilidad

ID	Requisito
RNF-11	Cada componente debe cumplir el principio de responsabilidad única, usar props para recibir datos y funciones, y manejar estado propio solo cuando sea necesario.
RNF-12	Las reglas de negocio deben implementarse como funciones puras en `utils/`, independientes del DOM y del almacenamiento.
RNF-13	Los componentes deben acceder a los datos únicamente a través de `data/db.js` y de los contextos.

ID	Requisito
RNF-14	El código se comenta en español, indicando propósito y reglas de negocio.

3.3.5 Portabilidad y usabilidad

ID	Requisito
RNF-15	Diseño responsivo con Bootstrap para pantallas desde 360 px, en móvil, tablet y escritorio.
RNF-16	Compatibilidad con las versiones actuales de Chrome, Firefox y Edge.
RNF-17	Los errores de formulario se muestran bajo cada campo y los campos obligatorios se marcan con asterisco.

3.4 Otros Requisitos

ID	Requisito
RNF-18	El código debe alojarse en un repositorio público de GitHub, con la migración a React en la rama `entrega-2-react` y el backend con la autenticación en la rama `entrega-3-backend`.
RNF-19	Cada integrante debe registrar commits propios con mensajes descriptivos.
RNF-20	El repositorio debe incluir un README con las instrucciones de ejecución, de pruebas y la estructura, y el backend su propio `README.md`.
RNF-21	La documentación se versiona en `docs/` junto al código.

3.5 Estrategia de pruebas

Aspecto	Definición
Herramientas	Frontend: Jasmine (pruebas), Karma (ejecución en Chrome sin ventana), webpack y Babel (compilación del JSX), `karma-coverage` con Istanbul (cobertura). Backend: `node:test` y el medidor de cobertura de Node
Comandos	En `frontend-react/`, `npm test` ejecuta todo y genera el reporte en `coverage/`, y `npm run test:watch` vuelve a ejecutar al guardar. En `backend/`, `npm test`. En la raíz, `npm test` corre el backend y luego el frontend
Niveles	Funciones puras (validaciones, carrito, pago, administración), capa de datos en sus dos modos, componentes (renderizado, props, estado, eventos), páginas y flujos completos; en el backend, la API completa contra una base en memoria (CRUD, autenticación y permisos)
Aislamiento	Cada prueba parte con `localStorage` y datos vacíos; se usan espías (`spyOn`, `createSpy`) para `console.error`, `window.print` y funciones entregadas por props, y un generador aleatorio inyectado para el código de envío. El servidor se simula con `spyOn(window, 'fetch')` en las pruebas del modo API, y el backend usa un reloj inyectable para probar el bloqueo por intentos
Criterio de aceptación	Todas las pruebas pasan y la cobertura de líneas supera el 90% en el frontend y en el backend
Resultado actual	Frontend: 188 pruebas, 96,08% de líneas. Backend: 33 pruebas, 98,37% de líneas
Evidencia	Documento de cobertura del testing (`docs/cobertura-testing.md`)